Analysis of Algorithms: COT 4400.002

By: Stefan Dedic, Gavin Wilson, Cameron Goz

Professor Fnu Kaleemunnisa

Final Project: Social Networks Using Core Graph Algorithms

1. Introduction

Graph algorithms play a vital role in modeling and analyzing real-world networks; examples include transportation systems, communication networks, and social platforms. In this project, we decided to model a social network as a graph and apply fundamental graph algorithms we learned in this course to analyze relationships between users.

The goal is to explore how traversal algorithms, shortest-path algorithms, and minimum spanning tree (MST) algorithms can be used to gain insights into network structure, connectivity, and relationship strength. By representing users as vertices, and connections between them as edges we can construct a *friendships graph* and simulate real-world scenarios such as connection paths, identifying influential users, and optimizing network structures.

2. Problem Statement

The problem we address is:

How can we model a social network using graph structures and apply graph algorithms to analyze user connectivity, relationship strength (measured by mutual connections), and optimal connections?

In real-world social networks:

- Users are connected through friendships
- Some relationships are stronger than others
- Users may be indirectly connected through mutual friends

We aim to:

- Traverse the network efficiently (BFS, DFS)
- Find optimal paths between users (Bellman-Ford)
- Identify efficient network structures (Kruskal's MST)

3. Graph Design

3.1 Vertices

Each vertex represents a user in the social network

3.2 Edges

Each edge represents a direct friendship between two users

3.3 Graph Type

The graph is undirected because friendships are mutual. If Alice is friends with Bob, Bob is also friends with Alice. The graph is weighted because each friendship edge has a cost based on mutual friends.

3.4 Edge Weight Formula

Friendship strength is measured by the number of mutual friends. Since Kruskal and Bellman-Ford operate on lower-cost paths/edges, we convert strength into cost:

$$Cost = 1 / (mutual_friends + 1)$$

A larger number of mutual friends produce a smaller cost, representing a stronger connection.

3.5 Dataset and Graph Construction

The dataset is synthetic, meaning we manually created users and friendship connections to represent a realistic social network.

The graph contains:

Number of vertices/users: UPDATE LATER

Number of edges/friendships: UPDATE LATER

Graph type: weighted, undirected

Dataset type: synthetic

The dataset was designed to include clusters of highly connected users and bridge users who connect different friend groups.

4. Algorithms

4.1 Breadth-First Search (BFS)

- Explores graph level by level
- Finds shortest path in terms of number of connections (edges)

Use In project + Pseudocode:

- Determines degrees of separation between users

```
BFS(Graph, start):
        if start not in Graph:
           raise error "Start node does not exist"

        create empty set visited
        create empty list order
        create queue and enqueue start
        add start to visited
        while queue is not empty:
              user ← dequeue from front of queue
              append user to order

              for each neighbor in Graph.get_neighbors(user):
                 if neighbor not in visited:
                    add neighbor to visited
                    enqueue neighbor into queue
        return order
```

Time Complexity:

- Theoretical: $O(V + E)$
  - Each vertex is visited once Þ $O(V)$
  - Each edge is explored once $\Rightarrow O(E)$
- Implementation (List Queue): $O(V^2 + E)$
  - In Python, lists are dynamic ranges; therefore, removing the first element requires shifting all elements left,

$$pop(0) = O(n) \setminus O(V) \times O(V) = O(V^2) \rightarrow \mathrm{O}(V^2 + \mathrm{E})$$

(Disclaimer: Unsure if usage of external libraries was permitted; therefore, usage of deque was omitted, this resulted in an increased time complexity for Implementation)

Best Case:

- Best Case Time Complexity would be $O(1)$ and occurs when:
  - Start node is isolated (contains no neighbors)
  - Only one vertex is processed

Worst Case:

- Worst Case Time Complexity in our implementation (due to lack of libraries) would be $O(V^2 + E)$ and occurs when:
  - Graph is fully connected
  - Queue grows large (Up to V)
  - Each pop(0) Costs $O(V)$

Space Complexity:

Space complexity of our implementation of BFS would be $O(V)$ because:

- *visited set* → $\mathrm{O(V)}$
- *queue* → $\mathrm{Up\ to\ O(V)}$
- *order list* → $\mathrm{O(V)}$

Where a Best-Case Space: $O(1)$ (only one node stored; isolated start).

Where a Worst-Case Space: $O(V)$ (Entire graphs stored).

4.2 Depth-First Search (DFS)

- Explores graph as deep as possible before backtracking

Use In project + Pseudocode:

- Determines degrees of separation between users

```
DFS(Graph, start):
        if start not in Graph:
            raise error "Start node does not exist"

        create empty set visited
        create empty list order
        define DFS_Helper(user):
            add user to visited
            append user to order

            for each neighbor in Graph.get_neighbors(user):
                if neighbor not in visited:
                    DFS_Helper(neighbor)

        call DFS_Helper(start)
        return order
```

Time complexity:

- $O(V + E)$, because:
- Each vertex is visited once → $\mathrm{O(V)}$
- Each edge is explored once → $\mathrm{O(E)}$

Best Case: Occurs when start node has no neighbors, resulting in $O(1)$

Worst Case: Occurs when entire graph is connected and DFS traverses all vertices and edges, resulting in $O(V + E)$

Space Complexity:

Space complexity of our implementation of DFS would be $O(V)$ because:

- *Visited set* → $\mathrm{O(V)}$
- *Order list* → $\mathrm{O(V)}$
- *Recursion Stack* → $\mathrm{up\ to\ O(V)}$

Best Case Time: Occurs when one node visited, minimal recursion depth, resulting in $O(1)$

Worst Case: Occurs when graph is like a chain, therefore, recursion depth becomes V, resulting in $\mathrm{O(V)}$

4.3 Bellman-Ford Algorithm

How it works + Pseudocode:

Bellman-Ford finds the shortest path from the starting vertex to all the other vertices in a weighted graph. In this project each vertex is a user, every edge shows a connection, and the edge weight represents the cost of moving through that friendship. Having a lower cost would mean the friendship connection is stronger based on the number of mutual friends there is. For the Bellman-Ford relaxes every edge, meaning it will go throughout each vertex and find cheaper paths. This takes a process of V-1 where V is the number of vertices.

Pseudocode:

```
Bellman-Ford(Graph, source):
    set all distances to infinity
    set distance of source = 0
    repeat (v - 1) times:
        for each edge(u,v):
            if distance[u] + weight < distance[v]:
                update distance[v]
    check for negative cycles:
        if any edge can still be relaxed:
            report negative cycle
    return distances
```

Time Complexity

- O(V x E)
    - The time complexity is looping through all of the edges E for V-1 times.

V being number of users

E being number of friendship connections

Space Complexity

- O(V)
    - The space complexity stores the distance value for each user and the predecessor value for each user.

Best Case

- The best case is O(E) because if there are no distances changing throughout the full pass of the edges, the algorithm can stop early. This usually happens when the shortest paths are found fast.

Worst Case

- O(VE) is the worst case because when the algorithm gas to go through V - 1 times before finding the final shortest path.

Use in Project

The use of Bellman-Ford in this project was to find the lowest cost path between users in a social network. This graph has mutual friends so to calculate the connection cost, this algorithm is there to find the strongest connection between two people, but in the fastest way possible.

4.4 Kruskal's Algorithm (MST)

- Find a minimum spanning tree in the synthetic social construct
- Finds the lowest-cost set of friendships that still connects all users without creating cycles
- Prioritizes stronger social connections first

How it works + Pseudocode:

Sorts all edges from lowest cost to highest. Then, check each edhe one at a time. If adding the edge connects two separate groups of users, the edge is added to the MST. If adding the edge would create a cycle, it is skipped.

This implementation uses union-find to keep track of which users are already connected. This prevents cycles and makes the algorithm efficient.

```
Kruskal(graph)
    sort edges by increasing cost
    make each user its own component
    mst = empty list
    total cost = 0

    for each edge in sorted edges:
    if the two users are in different components:
    add edge to mst
    add edge cost to total cost
    merge the two components
    return mst, total_cost
```

Time Complexity

- O(E log E); where E is the number of edges

The algorithm first sorts all edges (E) by cost (at time complexity E log E). After sorting it scans through edges once, for cost E. E log E grows faster than E, therefore the edge sorting dominates the runtime.

Space Complexity

- O(V + E); where V is the number of users and E is the number of edges

The algorithm stores the sorted edge list, parent directory, rank directory, and MST result

Best Case

The graph is sparse, such that it has fewer edges. This means theres fewer edges to sort, and the algorithm will run faster

Worst Case

The graph is dense, such that most users are connected to each other. This means more edges, which all have to be sorted and checked. This makes the algorithm take longer

Use in Project

Kruskal's gives a simplified version of the social network. It keeps the strongest connections needed to keep users connected, while removing unnecessary cycling formations and edges. It's used to show the most efficient overall structure of the friendship network.

5. Results

5.1 Traversal Results

BFS visits users level-by-level, showing degrees of separation from a starting user

DFS explores deep paths first, revealing connectivity structure and traversal paths

5.2 Shortest Path Results

- Bellman-Ford identifies indirect the lowest-cost path between users
- Unlike BFS, it considers edge weights (relationship strength), producing more meaningful paths in context of social connections

5.3 MST Results

Kruskal's algorithm produces a minimum spanning tree (MST) and connects all users with minimum total cost and eliminates redundant edges. This represents an optimized version of the social network.

5.4 Experimental Analysis & Comparison

Execution Time

BFS/DFS: Fastest $O(V + E)$

Kruskal (MST): Moderate $O(E \log E)$ (due to sorting edges)

Bellman-Ford: Slowest $O(VE)$

Memory Usage/Performance Observation

BFS: Uses a queue which has moderate memory usage

DFS: Uses recursion (call stack) therefore, relatively low memory

Bellman-Ford: Stores distance and predecessor tables which is moderate memory

Kruskal: Uses union-find which is efficient memory usage

Graph Size Variation

Small Graph (~15-20 Nodes)

All algorithms run very fast due to small amount of nodes

Large Graph (Simulated):

Bellman-Ford slows down significantly, BFS and DFS remain efficient,

Kruskal scales reasonably

Sparse vs Dense Graph Comparison

Spares Graph (few edges): All algorithms perform efficiently

Dense Graph (many edges): Bellman-ford becomes expensive, Kruskal slows due to increase sorting cost, BFS/DFS remain relatively efficient.

Example Results Table

Algorithm	Time Complexity	Performance	Best Use
BFS	(O(V + E))	Very Fast	Traversal
DFS	(O(V + E))	Very Fast	Traversal
Bellman-Ford	(O(VE))	Slow	Shortest Path
Kruskal (MST)	(O(E log E))	Moderate	Network Optimization

5.5 Discussion & Insights

BFS and DFS performed best due to:

- Linear time complexity $O(V + E)$
- Minimal computational overhead
- Being efficient for traversal

Bellman-Ford performed worst due to:

- Requires edge relaxation
- Higher time complexity $O\ (VE)$
- Performance degrades significantly as graph size increases

When to Use Each Algorithm:

BFS:

Shortest path in unweighted graphs

Level-order traversal

DFS:

Exploring connectivity

Detecting cycles or deep relationships

Bellman-Ford:
Weighted graphs

When negative weights are possible

Guaranteed correct shortest paths

Kruskal:

Building minimum spanning trees

Optimizing total network cost

6. Conclusion

In this project, we successfully modeled a social network as a weighted, undirected graph and applied multiple graph algorithms to analyze it.

Key takeaways:

- BFS is ideal for finding shortest paths in terms of connections
- DFS is useful for exploring network structure
- Bellman-Ford provides more meaningful paths by considering relationship strength
- Kruskal's algorithm helps identify efficient global connectivity

This project highlights how graph algorithms can be applied to real-world problems such as social network analysis, recommendation systems, and connectivity optimization.